Network Security · Intrusion Detection Systems · Cybersecurity

# NetWatchman

## Network Intrusion Detection System

A Python-based modular NIDS featuring 16 independent attack detectors, a live web dashboard with real-time SSE streaming, PCAP analysis, and full MITRE ATT&CK framework mapping.

| | |
|---|---|
| Project Type | Network Security / Cybersecurity Engineering |
| Language | Python 3.x |
| Core Libraries | Flask, Scapy, SQLite3, ReportLab |
| Frontend | HTML5, CSS3, JavaScript, Chart.js |
| Typography | IBM Plex Mono, Oxanium |
| Detectors | 16 Independent Attack Modules |
| MITRE Coverage | 12 Techniques Across 8 Tactics |

All PCAP files used for testing were sourced from publicly available repositories or synthesised in controlled lab environments for academic purposes only.

# Abstract

NetWatchman is a fully functional, Python-based Network Intrusion Detection System (NIDS) built from the ground up as a cybersecurity project. The system is designed to analyse pre-recorded PCAP network captures and replay them live through a Flask-powered web dashboard, enabling security analysts to inspect, classify, and export detected threats in real time.

At its core, NetWatchman contains sixteen modular, independently operating attack detectors covering a comprehensive range of threat categories: network reconnaissance (port scanning), layer-2 attacks (ARP spoofing), volumetric denial-of-service (SYN flood, ICMP flood), application-layer attacks (SQL injection, cross-site scripting, directory traversal), credential-based attacks (SSH brute force, FTP brute force), protocol abuse (DNS spoofing, Telnet detection), and exfiltration patterns (cleartext credential exposure, large data transfer detection, suspicious user agent fingerprinting, malicious IP feed lookup, and service version banner grabbing).

Every alert produced by the system is structured as a JSON object containing a severity classification (CRITICAL, HIGH, MEDIUM, LOW, or INFO), source and destination IP addresses, associated network ports, a human-readable description, a timestamp, and a direct mapping to the MITRE ATT&CK framework technique identifier. The web dashboard presents results through seven severity summary cards, an interactive filter toolbar with severity-coded buttons, a dynamic alert table whose message column renders inline MITRE chips and matched-credential chips, a slide-in detail side panel with MITRE ATT&CK context and a raw JSON inspector, and two Chart.js visualisations (donut and bar). Users can export findings as styled PDF reports via ReportLab or as CSV files for further processing.

A persistent Scan History module backed by SQLite automatically archives every scan result, allowing analysts to revisit, compare, and reload past analyses. The live replay mode streams packet-processing events to the browser via Server-Sent Events (SSE) with a five-level adjustable speed control, simulating a real-time threat monitoring experience complete with an animated live indicator, a progress bar with packet counter, and a live alert count ticker.

# Contents


**Abstract** **1**

**1 Introduction** **6**

1.1 Motivation and Context . . . 6

1.2 Project Goals . . . 6

1.3 Scope and Limitations . . . 7

1.4 Report Structure . . . 7

**2 System Architecture** **9**

2.1 High-Level Overview . . . 9

2.2 Directory Structure . . . 9

2.3 Supporting Root-Level Modules . . . 11

2.3.1 `alert_manager.py` — Alert Deduplication . . . 11

2.3.2 `mitre.py` — MITRE ATT&CK Enrichment Helper . . . 11

2.3.3 `enrichment.py` — AbuseIPDB Threat Intelligence . . . 11

2.3.4 `dashboard/static/style.css` — Dark Theme Stylesheet . . . 12

2.3.5 PCAP Analysis Mode . . . 12

2.3.6 Live Replay Mode . . . 13

2.4 Technology Stack . . . 13

2.5 Alert Schema . . . 14

**3 Attack Detectors** **15**

3.1 Detector 1 – Port Scan (`port_scan.py`) . . . 15

3.1.1 Attack Description . . . 15

3.1.2 Detection Logic . . . 15

3.2 Detector 2 – ARP Spoofing (`arp_spoof.py`) . . . 16

3.2.1 Attack Description . . . 16

3.2.2 Detection Logic . . . 16

3.3 Detector 3 – SSH Brute Force (`ssh_brute.py`) . . . 17

3.3.1 Attack Description . . . 17

3.3.2 Detection Logic . . . 17

3.4 Detector 4 – FTP Brute Force (`ftp_brute.py`) . . . 18

3.4.1 Attack Description . . . 18
3.4.2 Detection Logic . . . 18
3.5 Detector 5 – SYN Flood (syn_flood.py) . . . 18
3.5.1 Attack Description . . . 18
3.5.2 Detection Logic . . . 19
3.6 Detector 6 – ICMP Flood (icmp_flood.py) . . . 19
3.6.1 Attack Description . . . 19
3.6.2 Detection Logic . . . 19
3.7 Detector 7 – DNS Spoofing (dns_spoof.py) . . . 19
3.7.1 Attack Description . . . 20
3.7.2 Detection Logic . . . 20
3.8 Detector 8 – Cleartext Credentials (cleartext_creds.py) . . . 20
3.8.1 Attack Description . . . 20
3.8.2 Detection Logic . . . 20
3.9 Detector 9 – Directory Traversal (dir_traversal.py) . . . 21
3.9.1 Attack Description . . . 21
3.9.2 Detection Logic . . . 21
3.10 Detector 10 – Malicious IP Detection (malicious_ip.py) . . . 22
3.10.1 Attack Description . . . 22
3.10.2 Detection Logic . . . 22
3.11 Detector 11 – Service Version Detection (service_version.py) . . . 23
3.11.1 Attack Description . . . 23
3.11.2 Detection Logic . . . 23
3.12 Detector 12 – SQL Injection (sql_injection.py) . . . 23
3.12.1 Attack Description . . . 23
3.12.2 Detection Logic . . . 23
3.13 Detector 13 – Telnet Detection (telnet.py) . . . 24
3.13.1 Attack Description . . . 24
3.13.2 Detection Logic . . . 24
3.14 Detector 14 – Cross-Site Scripting (xss.py) . . . 25
3.14.1 Attack Description . . . 25
3.14.2 Detection Logic . . . 25
3.15 Detector 15 – Suspicious User Agents (suspicious_agents.py) . . . 25
3.15.1 Attack Description . . . 25
3.15.2 Detection Logic . . . 26
3.16 Detector 16 – Large Data Transfer (large_transfer.py) . . . 26
3.16.1 Attack Description . . . 26
3.16.2 Detection Logic . . . 26
3.17 Detector Summary Table . . . 27

**4 Web Dashboard** **28**
4.1 Overview . . . 28
4.2 PCAP Analyzer Tab . . . 28
4.2.1 File Upload Zone . . . 28
4.2.2 Navigation Tabs and Header . . . 29
4.2.3 Seven Severity Summary Cards . . . 29
4.2.4 Chart Visualisations . . . 29
4.2.5 Filter Toolbar . . . 30
4.2.6 Alert Table . . . 30
4.2.7 Message Column Chip System . . . 31
4.2.8 Slide-In Detail Panel . . . 31
4.2.9 Empty State . . . 32
4.2.10 Detector Warning Section . . . 32
4.3 Live Replay Tab . . . 32
4.3.1 SSE Streaming Architecture . . . 32
4.3.2 Speed Control Slider . . . 33
4.3.3 Live UI Elements During Replay . . . 34
4.3.4 Alert Row Prepending . . . 34
4.4 Scan History Tab . . . 35

**5 Export System and Scan History** **36**
5.1 Export System . . . 36
5.1.1 PDF Export (ReportLab) . . . 36
5.1.2 CSV Export . . . 37
5.2 Scan History (SQLite) . . . 37
5.2.1 Database Schema . . . 37
5.2.2 Auto-Save Behaviour . . . 37
5.2.3 History Retrieval and Deletion . . . 38

**6 MITRE ATT&CK Framework** **39**
6.1 Framework Overview . . . 39
6.2 ATT&CK Coverage in NetWatchman . . . 39
6.3 Tactics Distribution . . . 40

**7 Implementation Details** **41**
7.1 Packet Parsing Layer . . . 41
7.2 Detector Module Interface . . . 42
7.3 Flask Application Routes . . . 43
7.4 Frontend JavaScript Architecture . . . 43

**8 Testing and Results** **45**
8.1 Testing Methodology . . . . . 45
8.2 Detector Validation Results . . . . . 45
8.3 End-to-End Dashboard Validation . . . . . 46
8.4 Performance Observations . . . . . 47

**9 Conclusion** **48**
9.1 Summary . . . . . 48
9.2 Learning Outcomes . . . . . 48
9.3 Future Work . . . . . 49

**10 References** **50**

CHAPTER 1

# Introduction

## 1.1 Motivation and Context

Modern computer networks carry an ever-increasing volume of traffic, and with that growth comes a corresponding rise in malicious activity. Attackers probe open ports, flood services into unavailability, poison ARP caches to intercept traffic, brute-force authentication systems, inject malicious payloads into web applications, and exfiltrate sensitive data — often without being detected until significant damage has already occurred.

Network Intrusion Detection Systems (NIDS) sit at the heart of defensive network security. By passively monitoring network traffic and comparing observed behaviour against known attack signatures or behavioural baselines, a NIDS can alert security personnel to threats in progress or help analysts reconstruct past incidents from archived packet captures.

NetWatchman was created to build a working, complete NIDS from first principles — not as a wrapper around existing tools, but as a system where every detection module was designed, implemented, tested against real PCAP data, and integrated into a cohesive platform. The project covers the full lifecycle of a security alert: packet ingestion, protocol parsing, multi-detector analysis, alert generation, dashboard visualisation, historical storage, and report export.

## 1.2 Project Goals

The primary objectives of NetWatchman are:

1. **Detect real attack patterns** across 16 distinct threat categories using stateful, threshold-based, and signature-based detection logic.
2. **Operate on real PCAP files** captured from actual network activity, validating each detector against genuine malicious traffic.

3. **Present results visually** through a dark-themed professional web dashboard with charts, detail panels, and severity classification.
4. **Map every alert to MITRE ATT&CK** to connect detected behaviour to industry-standard threat intelligence taxonomy.
5. **Persist scan history** using SQLite so that past analyses remain accessible and comparable over time.
6. **Support report export** in both PDF and CSV formats for analyst workflows outside the dashboard.
7. **Provide live replay** of PCAP files with real-time SSE streaming to simulate a live monitoring environment.

## 1.3 Scope and Limitations

NetWatchman operates in passive analysis mode: it reads from PCAP files rather than capturing live traffic directly from a network interface (though this is architecturally straightforward to add via Scapy's `sniff()` function). The system is intended for academic demonstration and controlled-environment testing. Detection thresholds are calibrated for clarity and demonstration rather than production tuning, and the malicious IP detector relies on external connectivity to the Emerging Threats open blocklist.

## 1.4 Report Structure

- **Chapter 2** covers the overall system architecture, module layout, data flow, and technology stack.
- **Chapter 3** provides a detailed individual explanation of all 16 attack detectors.
- **Chapter 4** describes the web dashboard in full, including every UI component, the filter toolbar, the alert table message chip system, the slide-in detail panel, and all live-replay UI states.
- **Chapter 5** covers the export system and the persistent scan history feature.
- **Chapter 6** explores the MITRE ATT&CK framework and NetWatchman's mapping to it.
- **Chapter 7** discusses implementation details, technical decisions, and code design patterns.
- **Chapter 8** presents testing results, PCAP validation outcomes, and performance observations.

- **Chapter 9** concludes the project with reflections and future work.
- **Chapter 10** lists all references.

CHAPTER 2

# System Architecture

## 2.1 High-Level Overview

NetWatchman is a multi-layer Python application. The ingestion layer reads raw PCAP data; the parsing layer converts packets into structured dictionaries; the detection layer passes parsed packets through all 16 detector modules simultaneously; and the presentation layer surfaces results through a Flask web application backed by SQLite persistence.

```
[ PCAP File Upload ]
↓ sniffer.py - Scapy rdpcap() raw packet extraction
↓ parser.py - Structured packet dictionaries
↓ main.py - Orchestrator: 16 detectors run sequentially
↓ JSON Alert Stream
[ Flask app.py - REST API + SSE streaming ]
↓ dashboard.html - Chart.js, chips, detail panel, filter toolbar
↓ db.py - SQLite auto-save + history retrieval
↓ export_utils.py - ReportLab PDF + CSV generation
```

## 2.2 Directory Structure

```
netwatchman/
|-- detectors/                  # 16 attack detector modules
|    |-- __init__.py            # Makes detectors/ a Python package
|    |-- port_scan.py
|    |-- arp_spoof.py
|    |-- ssh_brute.py
|    |-- ftp_brute.py
|    |-- syn_flood.py
```

```
|    |-- icmp_flood.py           # Added as 16th detector (see note below)
|    |-- dns_spoof.py
|    |-- cleartext_creds.py
|    |-- dir_traversal.py
|    |-- malicious_ip.py
|    |-- service_version.py
|    |-- sql_injection.py
|    |-- telnet.py
|    |-- xss.py
|    |-- suspicious_agents.py
|    `-- large_transfer.py
|-- dashboard/                   # Flask web application
|    |-- app.py                  # Flask server + all API routes
|    |-- db.py                   # SQLite scan history layer
|    |-- export_utils.py         # PDF (ReportLab) + CSV export engine
|    |-- netwatchman_history.db  # Auto-created SQLite database
|    |-- static/
|    |    `-- style.css          # Dark cybersecurity theme stylesheet
|    `-- templates/
|         `-- dashboard.html     # Single-page web UI (SPA)
|-- pcap_samples/                # Test PCAP files used during development
|    |-- port_scan.pcap
|    |-- arp_spoof.pcap
|    |-- sql_injection.pcap
|    `-- ...
|-- output/
|    `-- reports/                # Generated PDF/CSV exports saved here
|-- sniffer.py                   # PCAP file reader (Scapy rdpcap)
|-- parser.py                    # Raw packet to structured dict converter
|-- main.py                      # Detector orchestrator entry point
|-- alert_manager.py             # Alert deduplication + MITRE enrichment
|-- mitre.py                     # MITRE ATT&CK technique lookup helper
|-- enrichment.py                # AbuseIPDB API threat intelligence
|-- requirements.txt
|-- .env                         # API keys (never committed to version control)
`-- README.md
```

Listing 2.1: NetWatchman Project Directory Structure

**Note on `icmp_flood.py` and `detectors/__init__.py`**

The project was originally designed with **15 detectors**. During the build session, `icmp_flood.py` was added as a **bonus 16th detector** — a decision that demonstrates the extensibility of the modular architecture: adding a new detector required only creating a single new file conforming to the `detect(packets)` interface and registering it in `main.py`. No other files needed modification.

The `detectors/__init__.py` file (empty) makes the `detectors/` directory a proper Python package, enabling clean imports of the form `from detectors import port_scan`.

## 2.3 Supporting Root-Level Modules

In addition to the core ingestion and detection pipeline, the project root contains three supporting modules:

### 2.3.1 `alert_manager.py` — Alert Deduplication

The alert manager receives the raw list of alerts produced by `main.py` and applies deduplication logic before passing results to the dashboard or storage layer. Because multiple detectors may observe the same underlying event (for example, a SYN flood producing hundreds of near-identical alerts within a single PCAP), the alert manager uses a stable hash of each alert's key fields to suppress duplicates and return a clean, non-redundant alert list.

### 2.3.2 `mitre.py` — MITRE ATT&CK Enrichment Helper

This module provides a lookup helper that maps technique IDs (e.g. `T1046`) to their full MITRE ATT&CK metadata. Each detector embeds its `mitre_id` and `mitre_name` directly in the alert JSON it produces, with `mitre.py` serving as a centralised reference for technique names and descriptions. The module includes graceful fallback: if a lookup fails for any reason, the main program never crashes — it returns a minimal dictionary containing only the technique ID and the ATT&CK URL.

### 2.3.3 `enrichment.py` — AbuseIPDB Threat Intelligence

The enrichment module queries the AbuseIPDB API (`https://www.abuseipdb.com/`) for every attacker IP found in the alerts, adding real-world threat intelligence context. This is separate from the `malicious_ip.py` detector (which uses the Emerging Threats

blocklist) — enrichment adds a confidence score and community report count to *existing* alerts rather than generating new ones.

| Property | Value |
|---|---|
| Endpoint | `https://api.abuseipdb.com/api/v2/check` |
| Method | GET |
| Auth | `Key: <api_key>` (stored in `.env`) |
| Free tier | 1,000 checks per day |
| Lookback | 90 days (`maxAgeInDays: 90`) |

Fields extracted per IP: `abuseConfidenceScore` (0–100), `totalReports`, `lastReportedAt`, `countryCode`, and `isp`. A score of 0–24 indicates low risk; 25–74 moderate; 75–100 high confidence malicious infrastructure.

The API key is stored in the `.env` file at the project root and loaded at runtime via `python-dotenv`. The `.env` file is listed in `.gitignore` and is never committed to version control.

### 2.3.4 `dashboard/static/style.css` — Dark Theme Stylesheet

All visual styling for the dashboard is defined in `static/style.css` as a standalone stylesheet, separate from the HTML template. It defines the complete dark cybersecurity colour palette using CSS custom properties (`var(-cyan)`, `var(-green)`, `var(-red)`, `var(-amber)`, `var(-text-dim)`, etc.), the IBM Plex Mono and Oxanium font imports, all component styles (cards, chips, tables, progress bars, the slide-in panel, the live dot animation keyframes, and the filter toolbar), and responsive breakpoints. Separating styles into `style.css` keeps `dashboard.html` focused on structure and JavaScript logic.

### 2.3.5 PCAP Analysis Mode

When a user uploads a PCAP file through the dashboard:

1. Flask saves the uploaded file to a temporary location and calls `sniffer.py`, which uses Scapy's `rdpcap()` to load all packets from disk into memory.
2. Each raw Scapy packet is passed to `parser.py`, which extracts the relevant fields into a Python dictionary: `timestamp`, `timestamp_raw` (float epoch), `src_ip`, `dst_ip`, `protocol`, `src_port`, `dst_port`, `payload`, `src_mac`, `dst_mac`, and where applicable `tcp_flags` and `arp_op`.
3. The full list of parsed packet dictionaries is passed sequentially to each of the 16 detector modules via `main.py`.

4. Each detector returns a (possibly empty) list of alert dictionaries. All alerts from all detectors are aggregated into a single JSON array.
5. Flask returns the complete alert array to the dashboard frontend. The frontend renders summary cards, charts, and the alert table. The current filename, scan mode, and timestamp are stored in JavaScript state variables (`currentSourceName`, `currentScanMode`, `currentScanTime`).
6. The scan result is automatically persisted to SQLite via `db.py` upon completion, without any user action required.

### 2.3.6 Live Replay Mode

In live replay mode, the same PCAP is processed but results are streamed incrementally to the browser:

1. The uploaded PCAP is pre-processed on upload into a session object stored in server-side memory, keyed by a UUID session ID. The response includes the session ID and total packet count.
2. The browser opens a persistent HTTP GET connection to `/replay/stream/<session_id>?speed=<1-5>`. The speed parameter is read from the slider before the connection is opened.
3. Flask uses a Python generator with `yield` to push Server-Sent Event messages over this connection. Three message types are emitted: `alert` (a new detection fired), `progress` (batch processed, percentage complete, packet counter, severity counts), and `done` / `stopped` (replay finished or cancelled by the Stop button).
4. The frontend appends each incoming alert to the table as a new row at the top, increments the live alert counter, updates the severity summary cards, and advances the progress bar.
5. Upon completion, the server emits the `scan_id` assigned by SQLite, which the frontend stores as `currentScanId` for subsequent export requests.

## 2.4 Technology Stack

| Component | Technology | Purpose |
|---|---|---|
| Packet ingestion | Scapy (rdpcap) | Load and iterate PCAP packets |
| Packet parsing | Python stdlib | Extract IP/TCP/UDP/ARP/ICMP fields |

| Component | Technology | Purpose |
|---|---|---|
| Detection engine | Python 3.x | 16 independent detector modules |
| Web server | Flask | REST API and SSE streaming |
| Frontend charts | Chart.js | Donut and bar chart visualisation |
| Typography | IBM Plex Mono | Monospace UI elements and code |
| Typography | Oxanium | Headings, badges, labels |
| Persistence | SQLite3 | Scan history storage |
| PDF export | ReportLab | Styled PDF report generation |
| CSV export | Python csv module | Flat-file alert export |
| Threat intelligence | Emerging Threats (HTTP) | Live malicious IP blocklist |
| Styling | Custom CSS | Dark cybersecurity colour palette |

## 2.5 Alert Schema

Every alert produced by any detector conforms to the following JSON schema:

```
{
  "type":        "string",   # Detector name, e.g. "port_scan"
  "severity":    "string",   # "CRITICAL"|"HIGH"|"MEDIUM"|"LOW"|"INFO"
  "src_ip":      "string",   # Source IP address
  "dst_ip":      "string",   # Destination IP address
  "src_port":    "integer",  # Source port (0 if not applicable)
  "dst_port":    "integer",  # Destination port (0 if not applicable)
  "description": "string",   # Human-readable alert summary
  "timestamp":   "string",   # ISO-8601 datetime string
  "mitre_id":    "string",   # MITRE ATT&CK technique, e.g. "T1046"
  "mitre_name":  "string"    # Full technique name
}
```

Listing 2.2: Standard Alert JSON Schema

CHAPTER 3

# Attack Detectors

NetWatchman contains sixteen independently operating detector modules. Each module receives the full list of parsed packet dictionaries, maintains its own internal state, applies detection logic, and returns a list of alert objects. This chapter explains every detector individually: the attack it targets, the detection logic, the data structures used, and the MITRE ATT&CK mapping.

## 3.1 Detector 1 – Port Scan (`port_scan.py`)

### 3.1.1 Attack Description

Port scanning is a reconnaissance technique in which an attacker sends probe packets to a range of TCP or UDP ports on a target host to determine which services are running. The most common method is the SYN scan (stealth scan / half-open scan): the attacker sends a TCP SYN packet to each port and observes whether the target responds with SYN-ACK (open) or RST (closed), without completing the three-way handshake. This makes the scan difficult to log on older systems but clearly visible at the packet level.

### 3.1.2 Detection Logic

The detector maintains a dictionary mapping each source IP to the set of unique destination ports it has sent SYN packets to. When the number of distinct destination ports from a single source exceeds 15, an alert is fired. Using a `set` rather than a counter prevents the same port being counted twice and ensures the alert reflects genuine port breadth.

```
from collections import defaultdict

port_map = defaultdict(set)    # { src_ip: {dst_port, ...} }
THRESHOLD = 15

for pkt in packets:
```

```
    if pkt['protocol'] == 'TCP' and 'S' in pkt.get('tcp_flags',''):
        port_map[pkt['src_ip']].add(pkt['dst_port'])
        if len(port_map[pkt['src_ip']]) >= THRESHOLD:
            # fire HIGH alert
```

Listing 3.1: Port Scan Core Logic

**MITRE ATT&CK Mapping**

T1046 – *Network Service Discovery — Tactic: TA0007 Discovery*

Adversaries may attempt to get a listing of services running on remote hosts to identify potential targets for lateral movement.

**Severity:** HIGH **Tested against:** Real Nmap SYN scan PCAP (100 ports, source IP 192.168.12.235)

## 3.2 Detector 2 – ARP Spoofing (arp_spoof.py)

### 3.2.1 Attack Description

ARP (Address Resolution Protocol) operates at Layer 2 to map IP addresses to MAC addresses. Because ARP is stateless and unauthenticated, an attacker on the same network segment can broadcast crafted ARP Reply packets that associate their own MAC address with the IP of another host (typically the default gateway). Victim machines update their ARP caches with the false mapping, causing all traffic intended for the gateway to flow through the attacker's machine — a classic Man-in-the-Middle (MITM) position enabling interception, modification, or dropping of traffic.

### 3.2.2 Detection Logic

The detector tracks a dictionary mapping each IP address to the set of MAC addresses that have claimed it across all ARP packets in the capture. If any single IP appears with more than one distinct MAC address, the ARP table is inconsistent — a definitive indicator of spoofing activity. This logic catches both gratuitous ARP floods and subtle targeted poisoning.

```
from collections import defaultdict

ip_to_macs = defaultdict(set)  # { ip: {mac1, mac2, ...} }

for pkt in packets:
    if pkt['protocol'] == 'ARP' and pkt.get('arp_op') == 2:
        ip_to_macs[pkt['src_ip']].add(pkt['src_mac'])
```

```
        if len(ip_to_macs[pkt['src_ip']]) > 1:
            # fire CRITICAL alert
```

Listing 3.2: ARP Spoof Core Logic

**MITRE ATT&CK Mapping**

T1557.002 – *Adversary-in-the-Middle: ARP Cache Poisoning*

*Tactic: TA0006 Credential Access / TA0009 Collection*

Adversaries may poison ARP caches to position themselves between communication flows to intercept data in transit.

**Severity:** CRITICAL **Tested against:** Real ARP poisoning PCAP (16,285 packets)

## 3.3 Detector 3 – SSH Brute Force (ssh_brute.py)

### 3.3.1 Attack Description

SSH brute force attacks involve an attacker repeatedly attempting to authenticate to a target SSH service (TCP port 22) using different credential combinations. Tools such as Hydra and Medusa can attempt hundreds of login combinations per minute. Successful compromise grants interactive shell access with the privileges of the targeted account.

### 3.3.2 Detection Logic

The detector uses a sliding time-window approach: a 60-second rolling window tracks TCP SYN packet counts from each source IP directed at port 22. When the count exceeds 20 within the window, a HIGH alert fires. When it exceeds 100, severity escalates to CRITICAL, indicating an automated high-speed tool rather than manual attempts.

```
from collections import defaultdict

ssh_attempts = defaultdict(list)  # { src_ip: [timestamp, ...] }
WINDOW = 60

for pkt in packets:
    if pkt['dst_port'] == 22 and pkt['protocol'] == 'TCP':
        src, now = pkt['src_ip'], pkt['timestamp_raw']
        ssh_attempts[src] = [t for t in ssh_attempts[src]
                             if now - t <= WINDOW]
        ssh_attempts[src].append(now)
        count = len(ssh_attempts[src])
        if count >= 100:   # fire CRITICAL
        elif count >= 20:  # fire HIGH
```

Listing 3.3: Sliding Window Brute Force Logic

**MITRE ATT&CK Mapping**
`T1110.001` – *Brute Force: Password Guessing* — *Tactic: TA0006 Credential Access*

**Severity:** HIGH / CRITICAL **Tested against:** Real SSH brute-force PCAP (228 attempts)

## 3.4 Detector 4 – FTP Brute Force (`ftp_brute.py`)

### 3.4.1 Attack Description

FTP (File Transfer Protocol) on TCP port 21 is a legacy protocol that transmits authentication credentials in cleartext and is frequently targeted by automated credential stuffing attacks. Many FTP servers lack rate-limiting, making them particularly attractive targets.

### 3.4.2 Detection Logic

Architecturally identical to the SSH detector, but targeting TCP port 21. The same 60-second sliding window and HIGH (20) / CRITICAL (100) thresholds apply. This parallel design keeps both modules easy to maintain and extend to other services.

**MITRE ATT&CK Mapping**
`T1110.001` – *Brute Force: Password Guessing* — *Tactic: TA0006 Credential Access*

**Severity:** HIGH / CRITICAL **Tested against:** Real FTP brute-force PCAP (332 attempts)

## 3.5 Detector 5 – SYN Flood (`syn_flood.py`)

### 3.5.1 Attack Description

A SYN flood is a volumetric Denial-of-Service attack exploiting the TCP three-way handshake. The attacker sends an overwhelming volume of TCP SYN packets to a target, typically with spoofed source IP addresses, causing the server to allocate half-open connection state entries. Each entry waits for a final ACK that never arrives. Once the connection table is exhausted, legitimate connections are refused.

### 3.5.2 Detection Logic

The detector counts inbound TCP SYN packets per destination IP within a rolling 5-second window (shorter than the brute-force window to catch volumetric bursts). Thresholds of 500 (HIGH) and 1,000 (CRITICAL) SYN packets per 5 seconds are applied. The short window is intentional: legitimate traffic rarely produces hundreds of half-open connections per second to a single host.

**MITRE ATT&CK Mapping**
`T1498.001` – *Network Denial of Service: Direct Network Flood*
*Tactic: TA0040 Impact*

**Severity:** HIGH / CRITICAL **Tested against:** Real SYN flood PCAP with spoofed source IPs

## 3.6 Detector 6 – ICMP Flood (`icmp_flood.py`)

### 3.6.1 Attack Description

An ICMP flood (ping flood) overwhelms a target with ICMP Echo Request packets, consuming bandwidth and CPU resources. The Smurf variant amplifies the effect by spoofing the victim's IP as the source and broadcasting to network amplifiers. Large ICMP payloads may also indicate a Ping of Death attempt targeting buffer overflow vulnerabilities in older network stacks.

### 3.6.2 Detection Logic

The detector counts ICMP Echo Request packets per source IP within a rolling time window. A high rate from a single source — or directed at a single destination — triggers alerts. The detector also checks for abnormally large ICMP payload sizes as a Ping of Death sub-indicator.

**MITRE ATT&CK Mapping**
`T1498` – *Network Denial of Service — Tactic: TA0040 Impact*

**Severity:** HIGH / CRITICAL **Tested against:** Real ICMP flood PCAP (961,000+ packets)

## 3.7 Detector 7 – DNS Spoofing (`dns_spoof.py`)

### 3.7.1 Attack Description

DNS spoofing (DNS cache poisoning) is an attack where a malicious actor intercepts DNS queries and returns fabricated responses, redirecting victims to attacker-controlled servers. This enables credential harvesting, malware delivery, and MITM attacks without requiring direct compromise of the target machine. DNS responses are trusted by default at the OS level, making this attack particularly effective.

### 3.7.2 Detection Logic

The detector monitors DNS responses (UDP port 53, DNS Answer records) and tracks which IP addresses have been returned for each queried domain. If the same domain returns different A-record IPs across different response packets in the same capture, a spoofing condition is flagged. Any DNS response that arrives without a corresponding query in the capture (unsolicited response) is also flagged.

**MITRE ATT&CK Mapping**
`T1557` – *Adversary-in-the-Middle*
*Tactic: TA0006 Credential Access / TA0009 Collection*

**Severity:** CRITICAL **Tested against:** Real DNS MITM PCAP

## 3.8 Detector 8 – Cleartext Credentials (`cleartext_creds.py`)

### 3.8.1 Attack Description

Several legacy protocols transmit usernames and passwords in plaintext: FTP sends credentials in response to `USER` and `PASS` commands; Telnet transmits all keystrokes unencrypted; basic HTTP authentication sends Base64-encoded (not encrypted) credentials in the `Authorization` header. Any attacker with passive packet capture capability can trivially extract these credentials.

### 3.8.2 Detection Logic

The detector inspects TCP packet payloads for plaintext credential markers: `USER`, `PASS`, `Authorization: Basic`, `login:`, and `password:` patterns. When matched, the actual credential value is extracted and included in the alert description to demonstrate the severity of the exposure. The search is case-insensitive to handle FTP server implementation variations.

```
PATTERNS = ['USER ', 'PASS ', 'Authorization: Basic',
            'login:', 'password:']

for pkt in packets:
    payload = pkt.get('payload', '')
    for pattern in PATTERNS:
        if pattern.lower() in payload.lower():
            value = payload.split(pattern, 1)[-1].strip()
            # fire HIGH alert, include value in description
```

Listing 3.4: Cleartext Credential Pattern Matching

**MITRE ATT&CK Mapping**
T1040 – *Network Sniffing — Tactic: TA0006 Credential Access*

**Severity:** HIGH **Tested against:** Real FTP session PCAP with visible USER/PASS commands

## 3.9 Detector 9 – Directory Traversal (dir_traversal.py)

### 3.9.1 Attack Description

Directory traversal (path traversal) is a web application attack where an attacker manipulates file path parameters in HTTP requests to access files outside the web root. Payloads such as `../../../etc/passwd` or URL-encoded variants like `%2e%2e%2f` attempt to read sensitive system files including password databases, configuration files, and private keys.

### 3.9.2 Detection Logic

The detector searches HTTP request payloads for traversal signatures: literal `../` sequences, URL-encoded equivalents (`%2e%2e%2f`), double-encoded variants, and common target paths such as `/etc/passwd`, `/etc/shadow`, and `/windows/system32`. Both the URL path and POST body are searched. URL decoding is applied before matching to catch encoded payloads.

**MITRE ATT&CK Mapping**
T1083 – *File and Directory Discovery — Tactic: TA0007 Discovery*

**Severity:** HIGH **Tested against:** Synthesised HTTP traversal PCAP

## 3.10 Detector 10 – Malicious IP Detection (malicious_ip.py)

### 3.10.1 Attack Description

Threat intelligence feeds maintain continuously updated lists of IP addresses associated with malicious activity: command-and-control servers, botnet nodes, TOR exit nodes, malware distribution infrastructure, and active scanning hosts. Detecting communication with these addresses is a high-confidence indicator of compromise.

### 3.10.2 Detection Logic

At runtime, the detector fetches the Emerging Threats open blocklist via HTTP (https://rules.emergingthreats.net/blockrules/compromised-ips.txt). This provides a live, up-to-date set of known-bad IPs. Every source and destination IP in the capture is checked against this set. A match triggers an immediate CRITICAL alert.

```python
import requests

def load_blocklist():
    url = "https://rules.emergingthreats.net/blockrules/compromised-ips.txt"
    resp = requests.get(url, timeout=10)
    return {l.strip() for l in resp.text.splitlines()
            if l and not l.startswith('#')}

blocklist = load_blocklist()
for pkt in packets:
    for ip in [pkt['src_ip'], pkt['dst_ip']]:
        if ip in blocklist:
            # fire CRITICAL alert
```

Listing 3.5: Live Threat Feed Integration

**MITRE ATT&CK Mapping**
T1071.001 – *Application Layer Protocol: Web Protocols*
*Tactic: TA0011 Command and Control*

**Severity:** CRITICAL **Tested against:** Live Emerging Threats feed + injected known-bad IPs

## 3.11 Detector 11 – Service Version Detection (`service_version.py`)

### 3.11.1 Attack Description

Many network services advertise their name and version in a banner string sent to connecting clients upon TCP connection establishment. SSH servers send strings such as `SSH-2.0-OpenSSH_7.4`; FTP servers send `220 vsftpd 2.3.4`; HTTP servers include version strings in response headers. Attackers collect these banners during reconnaissance to identify outdated, vulnerable software versions.

### 3.11.2 Detection Logic

The detector scans TCP payload strings for known service banner patterns using regular expressions, extracts the service name and version, and fires a MEDIUM alert (since banner grabbing is reconnaissance, not an active attack itself). The alert includes the full extracted banner string and the advertising IP address.

**MITRE ATT&CK Mapping**
`T1590.004` – *Gather Victim Network Information: Network Topology*
*Tactic: TA0043 Reconnaissance*

**Severity:** MEDIUM **Tested against:** Real SSH banner exchange PCAP

## 3.12 Detector 12 – SQL Injection (`sql_injection.py`)

### 3.12.1 Attack Description

SQL injection is among the most prevalent and dangerous web application vulnerabilities. Attackers inject malicious SQL syntax into input fields that are incorporated into database queries without proper sanitisation. Common payloads include tautologies (`' OR '1'='1`), UNION-based extraction, stacked queries (`; DROP TABLE`), and time-based blind injection (`SLEEP()`, `WAITFOR DELAY`). Successful exploitation can lead to authentication bypass, data exfiltration, and full database compromise.

### 3.12.2 Detection Logic

The detector searches HTTP payload strings against a curated list of SQL injection signatures covering the major attack families. Both the URL query string and POST body are searched. URL decoding is applied before matching.

```
SQL_SIGNATURES = [
    "' or '1'='1", "' or 1=1", "union select",
    "' --", "admin'--", "drop table", "insert into",
    "information_schema", "sleep(", "waitfor delay",
    "benchmark(", "0x", "' or 'x'='x"
]
```

Listing 3.6: SQL Injection Signature List

**MITRE ATT&CK Mapping**
`T1190` – *Exploit Public-Facing Application — Tactic: TA0001 Initial Access*

**Severity:** CRITICAL **Tested against:** Real SQL injection attack PCAP

## 3.13 Detector 13 – Telnet Detection (`telnet.py`)

### 3.13.1 Attack Description

Telnet (TCP port 23) is a decades-old remote terminal protocol that transmits all data — including login credentials, commands, and output — in unencrypted plaintext. It was superseded by SSH in the late 1990s. Any use of Telnet on a modern network is a significant security concern and may indicate legacy infrastructure, misconfiguration, or an attacker controlling compromised IoT devices (as seen in the Mirai botnet).

### 3.13.2 Detection Logic

The detection logic is deliberately simple: any TCP packet where either the source or destination port is 23 is flagged. The mere presence of Telnet traffic — regardless of direction — is inherently suspicious on a modern network. The detector differentiates between inbound Telnet (external party connecting to an internal Telnet server) and outbound Telnet (internal machine initiating Telnet to an external host).

**MITRE ATT&CK Mapping**
`T1021.004` – *Remote Services: SSH (Telnet is the insecure predecessor)*
*Tactic: TA0008 Lateral Movement*

**Severity:** HIGH **Tested against:** Real vintage Telnet session PCAP (circa 1999 traffic)

## 3.14 Detector 14 – Cross-Site Scripting (xss.py)

### 3.14.1 Attack Description

Cross-Site Scripting (XSS) is a web application vulnerability where an attacker injects malicious JavaScript or HTML into a page rendered in other users' browsers. Reflected XSS payloads appear in URLs; stored XSS payloads are persisted in databases and served to all visitors. Successful XSS enables session cookie theft, keylogging, credential harvesting, and browser exploitation.

### 3.14.2 Detection Logic

The detector searches HTTP request payloads for XSS signatures: `<script>` tags, `javascript:` protocol handlers, event handler attributes (`onerror=`, `onload=`, `onclick=`), `document.cookie` references, and common obfuscation patterns. URL decoding is applied before matching.

```
XSS_PATTERNS = [
    '<script', 'javascript:', 'onerror=', 'onload=',
    'onclick=', 'document.cookie', 'alert(', 'eval(',
    '<img src=x', '<svg onload', 'expression('
]
```

Listing 3.7: XSS Signature List

**MITRE ATT&CK Mapping**
T1190 – *Exploit Public-Facing Application — Tactic: TA0001 Initial Access*

**Severity:** CRITICAL **Tested against:** Synthesised XSS attack PCAP

## 3.15 Detector 15 – Suspicious User Agents (suspicious_agents.py)

### 3.15.1 Attack Description

HTTP clients identify themselves to web servers via the `User-Agent` header. Automated attack tools produce distinctive user agent strings: Nikto (web vulnerability scanner), sqlmap (SQL injection automation), Nmap HTTP scripts, Masscan, and Metasploit all use characteristic identifiers that are trivially detectable via string matching.

### 3.15.2 Detection Logic

The detector maintains a list of known-malicious user agent substrings and searches every HTTP request payload's `User-Agent` header against this list. A match fires a HIGH alert naming the detected tool and the communicating IP pair.

```
SUSPICIOUS_AGENTS = [
    'nikto', 'sqlmap', 'nmap', 'masscan', 'zgrab',
    'metasploit', 'hydra', 'dirb', 'gobuster',
    'python-requests', 'curl/', 'wget/', 'libwww-perl'
]
```

Listing 3.8: Suspicious User Agent Signatures

**MITRE ATT&CK Mapping**
`T1595.002` – *Active Scanning: Vulnerability Scanning*
*Tactic: TA0043 Reconnaissance*

**Severity:** HIGH **Tested against:** Synthesised scanner traffic PCAP

## 3.16 Detector 16 – Large Data Transfer (`large_transfer.py`)

### 3.16.1 Attack Description

Data exfiltration frequently manifests as anomalously large data transfers from internal hosts to external destinations. A single host transferring many megabytes to an unknown external IP within a short time window is a strong exfiltration indicator, particularly when the destination is not part of known corporate cloud services.

### 3.16.2 Detection Logic

The detector tracks the cumulative byte count of TCP/UDP payload data transferred between each unique (source IP, destination IP) pair within a rolling 60-second window. When the total exceeds 50 MB, a HIGH alert fires. The threshold is conservative for demonstration clarity and would be tuned per-environment in production.

```
from collections import defaultdict

transfer_vol = defaultdict(float)  # { (src, dst): bytes }

for pkt in packets:
    if pkt['protocol'] in ('TCP', 'UDP'):
```

```
        key = (pkt['src_ip'], pkt['dst_ip'])
        transfer_vol[key] += len(pkt.get('payload', ''))
        if transfer_vol[key] >= 50 * 1024 * 1024:
            # fire HIGH alert
```

Listing 3.9: Volume-Based Transfer Tracking

**MITRE ATT&CK Mapping**

T1048 – *Exfiltration Over Alternative Protocol*

*Tactic: TA0010 Exfiltration*

**Severity:** HIGH **Tested against:** Synthesised 50 MB TCP transfer PCAP

## 3.17 Detector Summary Table

| # | Detector | Protocol | MITRE ID | Max Severity |
|---|---|---|---|---|
| 1 | Port Scan | TCP | T1046 | HIGH |
| 2 | ARP Spoofing | ARP | T1557.002 | CRITICAL |
| 3 | SSH Brute Force | TCP/22 | T1110.001 | CRITICAL |
| 4 | FTP Brute Force | TCP/21 | T1110.001 | CRITICAL |
| 5 | SYN Flood | TCP | T1498.001 | CRITICAL |
| 6 | ICMP Flood | ICMP | T1498 | CRITICAL |
| 7 | DNS Spoofing | UDP/53 | T1557 | CRITICAL |
| 8 | Cleartext Creds | TCP | T1040 | HIGH |
| 9 | Dir Traversal | HTTP | T1083 | HIGH |
| 10 | Malicious IP | Any | T1071.001 | CRITICAL |
| 11 | Service Version | TCP | T1590.004 | MEDIUM |
| 12 | SQL Injection | HTTP | T1190 | CRITICAL |
| 13 | Telnet | TCP/23 | T1021.004 | HIGH |
| 14 | XSS | HTTP | T1190 | CRITICAL |
| 15 | Suspicious Agents | HTTP | T1595.002 | HIGH |
| 16 | Large Transfer | TCP/UDP | T1048 | HIGH |

CHAPTER 4

# Web Dashboard

## 4.1 Overview

The NetWatchman web dashboard is a single-page application (SPA) served by Flask and rendered entirely in the browser using vanilla HTML5, CSS3, and JavaScript with Chart.js for visualisations. It uses no frontend framework — all interactivity is implemented in plain ES6+. The application is styled in a dark cybersecurity aesthetic using two custom typefaces: **IBM Plex Mono** for all monospace interface elements (IP addresses, port numbers, code blocks, severity labels, chip text) and **Oxanium** for headings, card numbers, and display text. The colour palette uses deep near-black backgrounds with a cyan primary accent and severity-coded colours: red (CRITICAL), orange (HIGH), amber (MEDIUM), green (LOW), and purple (INFO).

Three modes are accessible through the top navigation bar: **PCAP Analyzer**, **Live Replay**, and **Scan History**.

## 4.2 PCAP Analyzer Tab

### 4.2.1 File Upload Zone

The upload area is a styled drag-and-drop zone. Users can either drag a capture file onto the zone or click to open the system file browser. The zone accepts three file extensions: `.pcap`, `.pcapng`, and `.cap`, covering all common capture formats produced by Wireshark, tcpdump, and other capture tools. Once a file is selected, its filename is displayed in a **green** `file-tag` chip and its size in muted monospace text, both rendered inline within the zone.

The *Run Analysis* button (`> Run Analysis`) submits the file to Flask's `/analyse` route via multipart form POST. While analysis is running, a dedicated `#progress-bar` / `#progress-fill` element beneath the button animates to indicate that the backend is

processing the PCAP. If the upload or analysis fails, an `#error-banner` div below the progress bar surfaces the error message in red. Both elements are hidden by default and conditionally shown.

### 4.2.2 Navigation Tabs and Header

The top navigation bar contains three mode tabs, each with a small coloured dot (`.tab-dot`) that acts as a status indicator for that mode:

- **Analyze PCAP** — green dot (`var(-green)`)
- **Live Replay** — no colour (default neutral)
- **Scan History** — amber dot (`var(-amber)`)

The page header above the tabs renders the NetWatchman logo alongside a dynamically injected detector count: `{{ detector_count }} Detectors`, where `detector_count` is a Flask template variable set to 16. This means the header count automatically updates if detectors are added or removed without touching the HTML.

### 4.2.3 Seven Severity Summary Cards

After analysis completes, seven metric cards are displayed across the top of the results section. Each card shows a single count and label:

| Card | ID | Colour | Metric |
|---|---|---|---|
| Total Alerts | `s-tot` | Cyan | All alerts combined |
| Packets | `s-pkt` | Cyan | Total packets in the PCAP |
| Critical | `s-critical` | Red | CRITICAL severity count |
| High | `s-high` | Orange | HIGH severity count |
| Medium | `s-medium` | Amber | MEDIUM severity count |
| Low | `s-low` | Green | LOW severity count |
| Info | `s-info` | Purple | INFO severity count |

Each card has a coloured 2px top border bar matching its severity colour and a hover lift transition. Cards update live during replay mode, incrementing in real time as each SSE alert message is received.

### 4.2.4 Chart Visualisations

Two Chart.js charts are rendered side by side below the summary cards:

- **Donut Chart** — Shows the proportional breakdown of alerts by severity level. Each segment is colour-coded (red/CRITICAL, orange/HIGH, amber/MEDIUM,

green/LOW, purple/INFO). Hovering over a segment displays the count and percentage. Only severity levels with at least one alert are included in the chart.

- **Bar Chart** — Shows the alert count grouped by detector type. Each bar represents one of the 16 detectors (only detectors with at least one alert are shown). This chart immediately reveals which attack types were most prevalent in the analysed PCAP. The x-axis labels use the detector's internal name string (e.g. `sql_injection`, `arp_spoof`).

Both Chart.js instances are destroyed and recreated on each new scan to prevent canvas state leaks from accumulating between analyses.

### 4.2.5 Filter Toolbar

The filter toolbar is a horizontal bar that appears above the alert table after a scan completes. It contains:

- **Severity filter buttons** — Six buttons labelled *All*, *Critical*, *High*, *Medium*, *Low*, and *Info*. Each button has a coloured active state matching its severity colour (e.g. the Critical button turns red with a red border and a red-tinted background when selected). Clicking a button hides all rows not matching that severity. Only one filter can be active at a time.
- **Visible count label** — A small monospace label (`#visible-count`) showing the number of alerts currently visible after applying the active filter and search query.
- **Search box** — A 240px-wide text input (`#search-box`) that performs a live substring search across all alert fields as the user types. The search box has a cyan focus glow. It combines with the severity filter: only rows matching both the active severity filter and the search query are shown.

When no rows match the current filter and search combination, a *"No alerts match the current filter"* message replaces the empty table.

### 4.2.6 Alert Table

The alert table is the primary data view. Each row represents one alert and contains five columns:

| Column | Content |
|---|---|
| Severity | Coloured badge: CRITICAL / HIGH / MEDIUM / LOW / INFO |
| Detector | Internal detector name (e.g. `port_scan`) |
| Source | `src_ip:src_port` in monospace |
| Destination | `dst_ip:dst_port` in monospace |
| Message | Rich chip-based message cell (see below) |

The table has a left border coloured by severity on each row (red for CRITICAL, orange for HIGH, etc.) providing an instant visual triage indicator.

### 4.2.7 Message Column Chip System

The Message column does not display raw text. Instead it renders a structured set of inline chips assembled from the alert's fields. The rendering priority is:

1. **Human-readable description** — The `description` field (or `detail` / `message` / `alert` as fallback) is displayed as plain text.
2. **MITRE ATT&CK chip** — If `mitre_id` and `mitre_name` are present, a `T1110.001 · Brute Force: Password Guessing` chip is rendered in cyan.
3. **Matched credential chip** — If the alert contains a `matched` field (used by the cleartext credentials detector to surface the extracted credential), an amber chip labelled `matched: USER bro` is rendered.
4. **Attempt / port count chips** — Numeric context chips (e.g. attempt counts, port counts) are rendered from relevant alert fields.
5. **Timestamp** — The alert timestamp is shown as inline muted monospace text.
6. **Raw JSON fallback** — If none of the above fields exist, the raw JSON object is rendered as a code block.

This chip system transforms raw JSON alert data into a human-readable, scannable message cell without losing any information.

### 4.2.8 Slide-In Detail Panel

Clicking any row in the alert table opens a slide-in detail panel from the right side of the screen. A full-page overlay backdrop darkens the rest of the dashboard, and the body scroll is locked while the panel is open. The panel contains:

- **Header** — Severity badge, detector name, source/destination IP pair.
- **Full description** — The complete `description` field rendered in a readable paragraph. Shown in muted colour if absent.
- **MITRE ATT&CK section** — A card showing the full technique ID (e.g. `T1046`), the technique name (e.g. *Network Service Discovery*), and a contextual description of what the technique represents in the adversary kill chain. This section is hidden if the alert has no `mitre_id`.

- **Extra fields section** — Any additional fields present in the alert JSON that are not part of the standard set of displayed fields (severity, type, IPs, ports, timestamp, description, MITRE) are rendered here as key-value pairs. This ensures no alert data is ever silently discarded by the UI.
- **Raw JSON inspector** — A collapsed code block showing the full raw alert JSON object, formatted with 2-space indentation. A *Show / Hide JSON* toggle button expands or collapses it. This is the raw data inspector for analysts who need to see all fields.

The panel is closed by clicking the overlay, clicking the close (X) button, or pressing Escape. On close, body scroll is restored.

### 4.2.9 Empty State

If a PCAP is successfully analysed but no detectors fired any alerts, the alert table area is replaced by a clean empty state: a large checkmark icon, the heading *"No Threats Detected"*, and the subtext *"All detectors ran clean against this capture."*. This makes it immediately clear that the absence of rows is a clean result, not a rendering error.

### 4.2.10 Detector Warning Section

Below the alert table, a *Detector Warnings* section (`#detector-errors`) is conditionally rendered if any detector module threw an exception during the analysis run. Each warning is shown as a card in the `#err-list` container identifying which detector failed and what the error was. This prevents silent failures going unnoticed in production use.

## 4.3 Live Replay Tab

The Live Replay tab provides a real-time analysis experience using the same upload zone, but processes the PCAP in a streaming fashion via Server-Sent Events.

### 4.3.1 SSE Streaming Architecture

The Live Replay tab presents a second drop zone styled with a play-button icon instead of an upload icon. Like the analyzer zone, it accepts `.pcap`, `.pcapng`, and `.cap` files. Once a file is selected, its filename is displayed in a **cyan** `file-tag` chip (distinct from the green chip in the analyze pane, giving an immediate visual cue of which mode is active).

Two buttons sit below the speed control:

- **Start Replay button** (`> Start Replay`, cyan styling) — initiates the upload and opens the SSE stream.

- **Stop button** (`[square] Stop`, red styling, `display:none` by default) — appears only while a replay is running and sends a POST to `/replay/stop/<session_id>` when clicked.

A `#replay-error` div below the buttons displays any upload or stream error in red, separate from the analyzer's own `#error-banner`.

After upload, Flask pre-processes the PCAP into a session object stored in server memory keyed by a UUID session ID. The browser then opens a long-lived GET connection to `/replay/stream/<session_id>?speed=<1-5>`. Flask uses a Python generator function with `yield` to push SSE-formatted JSON messages over this connection without closing it:

```
@app.route('/replay/stream/<session_id>')
def replay_stream(session_id):
    speed = int(request.args.get('speed', 3))
    def generate():
        for batch in process_session(session_id, speed):
            for alert in batch['alerts']:
                data = json.dumps({'type': 'alert', 'alert': alert})
                yield f"data: {data}\n\n"
            prog = json.dumps({'type':'progress','pct':batch['pct'],
                               'current':batch['current'],
                               'counts':batch['counts']})
            yield f"data: {prog}\n\n"
        done = json.dumps({'type': 'done', 'scan_id': save_to_db()})
        yield f"data: {done}\n\n"
    return Response(generate(), mimetype='text/event-stream')
```

Listing 4.1: Flask SSE Generator Pattern

### 4.3.2 Speed Control Slider

A five-position range input (`#speed-slider`, `min=1 max=5 value=3`) labelled *Replay Speed* controls replay speed before starting. The slider track is labelled **Slow** on the left end and **Fast** on the right end. A separate `#speed-label` span to the right of the slider shows the human-readable name of the currently selected level in cyan (e.g. *"Normal"*), updating live as the user drags. Each level adjusts two backend parameters: the number of packets processed per batch, and the inter-batch sleep delay:

| Level | Label | Batch Size | Delay |
|---|---|---|---|
| 1 | Slow | 5 pkts | 500 ms |
| 2 | Slower | 10 pkts | 250 ms |
| 3 | Normal | 20 pkts | 80 ms |
| 4 | Fast | 40 pkts | 30 ms |
| 5 | Fastest | 80 pkts | 10 ms |

The selected speed value is appended as a query parameter (`?speed=N`) to the SSE URL when the replay starts.

### 4.3.3 Live UI Elements During Replay

When a replay is in progress, the `#replay-status` block becomes visible and renders three live sub-elements:

- **Live indicator** (`.replay-live`) — An animated pulsing dot (`.live-dot`) next to the `#live-label` span, which reads `LIVE` in cyan while streaming. When replay finishes normally (`type='done'`), the dot's CSS class changes to `live-dot done` and the label switches to `COMPLETE` in green. If the Stop button is pressed (`type='stopped'`), the label switches to `STOPPED` in a neutral colour.
- **Packet counter** (`.replay-pkt-count`) — Displays the exact string *"Packets: X / Y"* where `#pkt-current` and `#pkt-total` are updated from the `current` field in each `progress` SSE message. The total is set from the session upload response before the stream opens.
- **Alert counter** (`.replay-alert-count`) — Displays the exact string *"Alerts detected: X"* where `#alert-live-count` increments by one each time an `alert` SSE message is received.

Above the status block, a `#replay-prog-bar` / `#replay-prog-fill` progress bar advances as each batch completes, driven by the `pct` value in `progress` SSE messages. The Stop button (`display:none` by default) becomes visible as soon as the stream opens and hides again when the replay ends.

### 4.3.4 Alert Row Prepending

During live replay, new alert rows are *prepended* to the top of the alert table as they arrive (newest first), giving the experience of watching alerts come in as a live feed rather than accumulating at the bottom.

## 4.4 Scan History Tab

The Scan History tab (`#pane-history`) displays a chronological list of all scans persisted in SQLite. The section header row contains two elements side by side: the *Scan History* label with an inline `.hist-count-badge` showing the total number of saved scans as a small numbered badge, and a **Clear All** button (`.hist-clear-btn`) on the right that wipes the entire scan history from SQLite and re-renders the list as empty.

When no scans have been saved yet, a `#hist-empty` empty state is displayed with a neutral icon and instructional text.

Each history entry card shows the PCAP filename, the scan date and time, the total alert count, and a per-severity breakdown (Critical / High / Medium / Low). Clicking any entry loads that scan's full alert JSON from the `/history/<id>` endpoint and renders it into the dashboard exactly as a fresh scan would — charts, summary cards, filter toolbar, and alert table all populate from the archived data.

A dismissable **history banner** appears at the top of the results section when a session has been loaded from history rather than from a fresh scan, making it clear to the analyst that they are viewing archived data. Individual history entries can also be deleted via a per-card delete button, which sends a DELETE request to `/history/<id>` and removes that row from SQLite.

CHAPTER 5

# Export System and Scan History

## 5.1 Export System

The export system is implemented in `export_utils.py` and exposes two functions called from Flask routes when the user activates the *Export* dropdown in the dashboard toolbar.

### 5.1.1 PDF Export (ReportLab)

The PDF report is generated using ReportLab's Platypus high-level document layout engine. The Flask route receives the full alert JSON array via POST body, along with the scan metadata (filename, scan time) stored in the frontend's JavaScript state variables (`currentSourceName`, `currentScanTime`). The generated document includes:

- **Header section** — NetWatchman branding, scan metadata (filename, date/time, total packet count, total alert count), and a severity summary breakdown table.
- **Alert table** — Every alert as a formatted row: severity badge (colour-coded), detector type, source/destination IP pair, MITRE technique ID, and description text. Row background colours are applied by severity using ReportLab's table style system.
- **Footer** — Page numbers and PDF generation timestamp on every page.

The PDF uses a dark background palette (`RGB(30, 40, 50)`) consistent with the dashboard aesthetic, with severity colours applied to row highlighting and badge text. The file is returned to the browser as an attachment download.

**Note on PDF rendering:** When viewed in built-in browser PDF viewers (Chrome, Edge), certain row backgrounds may show faint horizontal artefacts due to anti-aliasing. These are render-engine artefacts and disappear when the PDF is opened in Adobe Acrobat Reader or Foxit PDF Reader.

### 5.1.2 CSV Export

The CSV export produces a flat, standards-compliant file using Python's built-in `csv.DictWriter`. Every alert is written as one row. The columns are:

`severity, type, src_ip, dst_ip, src_port, dst_port, mitre_id, mitre_name, description, timestamp`

The file is returned with `Content-Disposition: attachment; filename=netwatchman_export.csv` headers. This format is directly importable into SIEM platforms, Microsoft Excel, and Python-based data analysis tools such as pandas.

## 5.2 Scan History (SQLite)

The scan history system is implemented in `db.py` using Python's built-in `sqlite3` module. No external database dependency is required — the SQLite file `netwatchman_history.db` is created automatically in the `dashboard/` directory on first run.

### 5.2.1 Database Schema

```sql
CREATE TABLE IF NOT EXISTS scans (
    id           INTEGER PRIMARY KEY AUTOINCREMENT,
    filename     TEXT    NOT NULL,
    scanned_at   TEXT    NOT NULL,      -- ISO-8601 datetime string
    alert_count  INTEGER NOT NULL,
    critical     INTEGER DEFAULT 0,
    high         INTEGER DEFAULT 0,
    medium       INTEGER DEFAULT 0,
    low          INTEGER DEFAULT 0,
    alerts_json  TEXT    NOT NULL       -- Full JSON alert array
);
```

Listing 5.1: SQLite Scan History Schema

### 5.2.2 Auto-Save Behaviour

Every time a PCAP analysis completes — whether in analyzer mode (synchronous) or replay mode (via SSE) — the result is automatically persisted via `db.save_scan()` without any user action. In replay mode, the assigned SQLite row ID is included in the final `done` SSE message as `scan_id` so the frontend can store it in `currentScanId` for subsequent export requests.

### 5.2.3 History Retrieval and Deletion

The `/history` route returns a JSON array of scan summaries (filename, scanned_at, alert_count, and per-severity counts) without the full alert JSON blobs, keeping the list response lightweight. When a user clicks a history entry, a follow-up request to `/history/<id>` fetches the complete `alerts_json` blob. The frontend then calls the same rendering functions used by a fresh scan. History entries can also be deleted individually via a DELETE request to `/history/<id>`, which removes the row from SQLite and re-renders the history list.

CHAPTER 6

# MITRE ATT&CK Framework

## 6.1 Framework Overview

The MITRE ATT&CK (Adversarial Tactics, Techniques, and Common Knowledge) framework is a globally accessible knowledge base of adversary tactics and techniques based on real-world observations of cyberattacks. It is maintained by MITRE Corporation and widely used by security teams for threat modelling, detection engineering, red team planning, and incident response.

The framework organises adversary behaviour into a matrix of **Tactics** (the adversary's goal — the *why*) and **Techniques** (the specific method — the *how*). Each technique has a unique identifier (e.g. T1046) and may have sub-techniques denoted with a decimal suffix (e.g. T1110.001 is the "Password Guessing" sub-technique of T1110 "Brute Force").

## 6.2 ATT&CK Coverage in NetWatchman

| Detector | Tactic | ID | Technique Name |
|---|---|---|---|
| Port Scan | Discovery | T1046 | Network Service Discovery |
| ARP Spoofing | Credential Access | T1557.002 | ARP Cache Poisoning |
| SSH Brute Force | Credential Access | T1110.001 | Password Guessing |
| FTP Brute Force | Credential Access | T1110.001 | Password Guessing |
| SYN Flood | Impact | T1498.001 | Direct Network Flood |
| ICMP Flood | Impact | T1498 | Network Denial of Service |
| DNS Spoofing | Collection | T1557 | Adversary-in-the-Middle |
| Cleartext Creds | Credential Access | T1040 | Network Sniffing |
| Dir Traversal | Discovery | T1083 | File and Directory Discovery |
| Malicious IP | Command & Control | T1071.001 | Web Protocols |
| Service Version | Reconnaissance | T1590.004 | Network Topology |

| Detector | Tactic | ID | Technique Name |
|---|---|---|---|
| SQL Injection | Initial Access | T1190 | Exploit Public-Facing App |
| Telnet | Lateral Movement | T1021.004 | Remote Services |
| XSS | Initial Access | T1190 | Exploit Public-Facing App |
| Suspicious Agents | Reconnaissance | T1595.002 | Vulnerability Scanning |
| Large Transfer | Exfiltration | T1048 | Exfil Over Alt. Protocol |

## 6.3 Tactics Distribution

NetWatchman's 16 detectors span eight MITRE ATT&CK Enterprise tactics:

- **TA0043 Reconnaissance** — Detectors 11 (service banner grabbing), 15 (vulnerability scanner agents)
- **TA0001 Initial Access** — Detectors 12, 14 (SQL injection, XSS exploiting public-facing applications)
- **TA0006 Credential Access** — Detectors 2, 3, 4, 8 (ARP poisoning, SSH/FTP brute force, cleartext sniffing)
- **TA0007 Discovery** — Detectors 1, 9 (port scanning, directory traversal)
- **TA0008 Lateral Movement** — Detector 13 (Telnet usage as insecure remote service)
- **TA0009 Collection** — Detector 7 (DNS spoofing enabling MITM data collection)
- **TA0010 Exfiltration** — Detector 16 (large volume transfer detection)
- **TA0011 Command and Control** — Detector 10 (communication with known C2 IPs)
- **TA0040 Impact** — Detectors 5, 6 (SYN flood, ICMP flood causing service disruption)

CHAPTER 7

# Implementation Details

## 7.1 Packet Parsing Layer

parser.py is the normalisation layer between raw Scapy packet objects and the rest of the system. It accepts a single Scapy packet and returns a dictionary with a consistent schema regardless of the underlying protocol:

```
from scapy.all import IP, TCP, UDP, ARP, ICMP, Raw

def parse_packet(pkt):
    parsed = {
        'timestamp':     str(pkt.time),
        'timestamp_raw': float(pkt.time),
        'src_ip': '', 'dst_ip': '',
        'protocol': 'OTHER',
        'src_port': 0, 'dst_port': 0,
        'payload': '',
        'src_mac': pkt.src if hasattr(pkt,'src') else '',
        'dst_mac': pkt.dst if hasattr(pkt,'dst') else '',
    }
    if ARP in pkt:
        parsed.update({'protocol':'ARP','src_ip':pkt[ARP].psrc,
                       'dst_ip':pkt[ARP].pdst,'src_mac':pkt[ARP].hwsrc,
                       'arp_op':pkt[ARP].op})
    elif IP in pkt:
        parsed['src_ip'] = pkt[IP].src
        parsed['dst_ip'] = pkt[IP].dst
        if TCP in pkt:
            parsed.update({'protocol':'TCP','src_port':pkt[TCP].sport,
                           'dst_port':pkt[TCP].dport,
                           'tcp_flags':str(pkt[TCP].flags)})
        elif UDP in pkt:
```

```
            parsed.update({'protocol':'UDP','src_port':pkt[UDP].sport,
                           'dst_port':pkt[UDP].dport})
        elif ICMP in pkt:
            parsed['protocol'] = 'ICMP'
        if Raw in pkt:
            try:
                parsed['payload'] = pkt[Raw].load.decode('utf-8', errors='ignore')
            except Exception:
                parsed['payload'] = ''
    return parsed
```

Listing 7.1: parser.py Normalisation Logic

## 7.2 Detector Module Interface

All 16 detectors follow the same interface contract: a single function `detect(packets)` that accepts the full list of parsed packet dictionaries and returns a list of alert dictionaries (empty list if nothing was detected). This uniformity allows `main.py` to orchestrate all detectors with a simple loop:

```
from detectors import (
    port_scan, arp_spoof, ssh_brute, ftp_brute, syn_flood,
    icmp_flood, dns_spoof, cleartext_creds, dir_traversal,
    malicious_ip, service_version, sql_injection, telnet,
    xss, suspicious_agents, large_transfer
)

ALL_DETECTORS = [
    port_scan, arp_spoof, ssh_brute, ftp_brute, syn_flood,
    icmp_flood, dns_spoof, cleartext_creds, dir_traversal,
    malicious_ip, service_version, sql_injection, telnet,
    xss, suspicious_agents, large_transfer
]

def run_all(packets):
    alerts = []
    for detector in ALL_DETECTORS:
        try:
            alerts.extend(detector.detect(packets))
        except Exception as e:
            # Logged to detector warnings section
            alerts.append({'type': detector.__name__, 'error': str(e)})
    return alerts
```

Listing 7.2: Detector Orchestration in main.py

## 7.3 Flask Application Routes

| Route | Method | Description |
|---|---|---|
| `/` | GET | Serve the dashboard HTML |
| `/analyse` | POST | Upload and analyse PCAP, return alerts JSON |
| `/replay/upload` | POST | Upload PCAP, create session, return session ID |
| `/replay/stream/<session>` | GET | SSE stream: alert, progress, done messages |
| `/replay/stop/<session>` | POST | Cancel active replay, emit stopped message |
| `/export/pdf` | POST | Build PDF from posted alerts JSON |
| `/export/csv` | POST | Build CSV from posted alerts JSON |
| `/history` | GET | List all scan summaries from SQLite |
| `/history/<id>` | GET | Fetch full alerts JSON for one scan |
| `/history/<id>` | DELETE | Remove a scan entry from SQLite |

## 7.4 Frontend JavaScript Architecture

The dashboard frontend is entirely contained in `dashboard.html` as a single-page application. Key subsystems:

- **Global state** — A `allAlerts` array holds the current scan's alert data. Auxiliary state variables track `currentSourceName`, `currentScanMode` (`'live'` or `'analyse'`), `currentScanTime`, and `currentScanId`. All UI components read from these.
- **Chart management** — Two Chart.js instances are stored in module-level variables. On each new scan, both instances are destroyed (via `.destroy()`) before being recreated, preventing memory leaks and canvas state contamination.
- **SSE client** — An `EventSource` object is opened against the replay stream endpoint. Three message types (`alert`, `progress`, `done`/`stopped`) are dispatched via the `onmessage` callback. The `EventSource` is closed on `done`, `stopped`, or when the user navigates away.

- **Detail panel** — The slide-in panel is toggled by adding/removing CSS classes (`.open`) on both the panel element and the overlay. Body scroll is locked via `document.body.style.overflow = 'hidden'` and restored on close.
- **Filter and search** — Filter button clicks set a module-level `activeSeverity` variable and call a shared `applyFilters()` function. The search box `oninput` handler also calls `applyFilters()`. The function iterates all table rows and sets `display: none` or `display: ''` based on the combined severity and search condition.
- **History tab** — Clicking a history entry fetches from `/history/<id>` and passes the result to the same rendering pipeline as a fresh scan, then shows the dismissable history banner via a CSS class toggle.

CHAPTER 8

# Testing and Results

## 8.1 Testing Methodology

Each of the 16 detectors was validated against one or more PCAP files before integration. Three test categories were used:

1. **Real-world PCAPs** — Publicly available captures from repositories such as Wireshark's sample captures, Malware Traffic Analysis, and PCAPDroid datasets. These contain genuine malicious traffic captured in real network environments.
2. **Synthesised PCAPs** — For attack types where authentic captures were not available, PCAPs were generated programmatically using Scapy's `wrpcap()` to create packets matching the expected attack pattern.
3. **Live feed testing** — The malicious IP detector was tested by injecting known-bad IPs from the live Emerging Threats blocklist into synthesised traffic, verifying both the HTTP fetch and set membership lookup functioned correctly.

## 8.2 Detector Validation Results

| # | Detector | Test PCAP | Result |
|---|---|---|---|
| 1 | Port Scan | Nmap SYN scan (100 ports, 192.168.12.235) | **PASS** |
| 2 | ARP Spoofing | ARP poisoning capture (16,285 packets) | **PASS** |
| 3 | SSH Brute Force | Hydra SSH attack (228 attempts) | **PASS** |
| 4 | FTP Brute Force | Hydra FTP attack (332 attempts) | **PASS** |
| 5 | SYN Flood | Spoofed SYN flood capture | **PASS** |
| 6 | ICMP Flood | Ping flood capture (961,000+ ICMP packets) | **PASS** |
| 7 | DNS Spoofing | DNS MITM capture (conflicting A records) | **PASS** |
| 8 | Cleartext Creds | FTP session (USER/PASS visible in payload) | **PASS** |

| # | Detector | Test PCAP | Result |
|---|---|---|---|
| 9 | Dir Traversal | Synthesised HTTP GET with `../` payload | **PASS** |
| 10 | Malicious IP | Live blocklist + injected bad IP | **PASS** |
| 11 | Service Version | Real SSH banner exchange capture | **PASS** |
| 12 | SQL Injection | Real SQLi attack traffic capture | **PASS** |
| 13 | Telnet | Vintage Telnet session PCAP (circa 1999) | **PASS** |
| 14 | XSS | Synthesised HTTP POST with script tag | **PASS** |
| 15 | Suspicious Agents | Synthesised Nikto/sqlmap user agent traffic | **PASS** |
| 16 | Large Transfer | Synthesised 50 MB TCP payload transfer | **PASS** |

All 16 detectors passed their respective validation tests, firing the correct alert type and severity for the expected attack traffic with no false negatives on the test captures.

## 8.3 End-to-End Dashboard Validation

The complete pipeline was tested by uploading each test PCAP through the web dashboard and verifying:

- Alert counts in all seven summary cards matched the raw JSON output from `main.py`.
- Donut chart segments correctly reflected the severity distribution.
- Bar chart bars correctly reflected per-detector alert counts.
- Message column chips rendered MITRE IDs and matched-credential values correctly.
- Severity filter buttons correctly showed and hid rows.
- The live search box correctly filtered rows on input.
- Clicking a row opened the slide-in panel with correct MITRE context.
- The raw JSON inspector in the panel showed complete, valid JSON.
- PDF exports were correctly generated with matching alert counts.
- CSV exports contained all expected columns and rows.
- Scan history entries were auto-created and could be reloaded cleanly.
- The history banner was shown on reload and was dismissable.
- Live replay streamed alerts progressively at all five speed levels.
- The progress bar, packet counter, and live alert count all updated correctly during replay.

- The Stop button successfully cancelled a mid-replay session.
- The live dot transitioned correctly through LIVE, COMPLETE, and STOPPED states.
- The footer hint text updated during and after replay.
- Clean PCAPs triggered the "No Threats Detected" empty state correctly.

## 8.4 Performance Observations

Analysis time scales approximately linearly with packet count. Representative timings:

| PCAP Size | Packet Count | Analysis Time |
|---|---|---|
| Small | ∼5,000 packets | under 1 second |
| Medium | ∼50,000 packets | 2–5 seconds |
| Large | ∼500,000 packets | 15–30 seconds |
| Very Large | ∼961,000+ packets | 45–60 seconds |

The primary bottleneck is Scapy's Python-layer packet parsing rather than the detection logic itself. The 16 detectors combined add negligible overhead compared to the parse time.

CHAPTER 9

# Conclusion

## 9.1 Summary

NetWatchman is a complete, working Network Intrusion Detection System built from first principles in Python. Sixteen independent attack detectors were designed, implemented, and individually validated against real PCAP network captures, covering the attack lifecycle from initial reconnaissance and credential theft through denial-of-service, web application exploitation, and data exfiltration — every alert grounded in the MITRE ATT&CK framework.

The Flask web dashboard transforms raw alert data into a fully featured analysis environment. Seven severity summary cards, two Chart.js visualisations, a colour-coded filter toolbar with live search, a chip-based message column, a slide-in detail panel with MITRE ATT&CK context and a raw JSON inspector, and a live replay mode with SSE streaming, speed control, animated live indicators, progress tracking, and a Stop button — all combine into a tool that looks and functions like a professional SOC-grade application.

The export system produces styled PDF reports and analyst-ready CSV files. The SQLite-backed scan history ensures no result is ever lost, with auto-save on every scan and single-click history reload.

## 9.2 Learning Outcomes

This project provided practical experience in:

- **Network protocol analysis** — Understanding TCP/IP, ARP, DNS, ICMP, and application-layer protocols at the packet level using Scapy.
- **Intrusion detection methodology** — Threshold-based, signature-based, and live feed-based detection strategies.

- **Web application development** — Full-stack Flask with REST APIs, SSE streaming, file upload handling, and a dynamic single-page frontend.
- **UI engineering** — Building a chip-based message rendering system, a slide-in panel with overlay and scroll lock, dynamic severity filtering and live search, and multiple distinct animated states for UI indicators.
- **Threat intelligence integration** — Consuming live IP reputation feeds and mapping detections to MITRE ATT&CK identifiers.
- **Data persistence and export** — SQLite for structured storage, ReportLab for programmatic PDF generation, and the Python csv module for flat-file export.
- **Security tool design** — Architecting a modular system where individual components can be tested, replaced, and extended independently with a shared interface contract.

## 9.3 Future Work

1. **Live capture mode** — Integrate Scapy's `sniff()` to analyse traffic from a live network interface in addition to pre-recorded PCAPs.
2. **Machine learning anomaly detection** — Train a baseline model on normal traffic and flag statistical deviations, enabling detection of novel attacks beyond the current signature and threshold set.
3. **Alert deduplication** — Implement suppression logic to avoid flooding the dashboard with repeated alerts for the same ongoing attack within a single scan.
4. **Email and webhook alerting** — Push CRITICAL alerts to email or Slack in real time, enabling operator response without keeping the dashboard open.
5. **Multi-file analysis** — Support analysing and comparing multiple PCAPs in a single dashboard session.
6. **Containerisation** — Package the application as a Docker image for reproducible deployment across environments.
7. **Performance optimisation** — Replace Scapy's Python-layer parser with a C-based alternative (dpkt or Rust bindings) for significantly faster large-file analysis.
8. **User authentication** — Add login and role-based access control for multi-analyst deployments.

CHAPTER 10

# References


[1] MITRE Corporation. *MITRE ATT&CK Enterprise Matrix*. Available at: https://attack.mitre.org/matrices/enterprise/ [Accessed 2026].

[2] Scapy Project. *Scapy Documentation*. Available at: https://scapy.readthedocs.io/ [Accessed 2026].

[3] Pallets Projects. *Flask Documentation*. Available at: https://flask.palletsprojects.com/ [Accessed 2026].

[4] Chart.js Contributors. *Chart.js Documentation*. Available at: https://www.chartjs.org/docs/ [Accessed 2026].

[5] ReportLab. *ReportLab User Guide*. Available at: https://docs.reportlab.com/ [Accessed 2026].

[6] Proofpoint / Emerging Threats. *Emerging Threats Open Ruleset*. Available at: https://rules.emergingthreats.net/ [Accessed 2026].

[7] Python Software Foundation. *sqlite3 – DB-API 2.0 interface for SQLite*. Available at: https://docs.python.org/3/library/sqlite3.html [Accessed 2026].

[8] Python Software Foundation. *csv – CSV File Reading and Writing*. Available at: https://docs.python.org/3/library/csv.html [Accessed 2026].

[9] Mozilla Developer Network. *Server-Sent Events*. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events [Accessed 2026].

[10] Sanders, C. and Smith, J. *Applied Network Security Monitoring: Collection, Detection, and Analysis*. Syngress, 2013.

[11] Bejtlich, R. *The Practice of Network Security Monitoring*. No Starch Press, 2013.

[12] OWASP Foundation. *OWASP Top Ten*. Available at: https://owasp.org/www-project-top-ten/ [Accessed 2026].

[13] Nmap Project. *Nmap Network Scanning*. Available at: https://nmap.org/book/ [Accessed 2026].

[14] Wireshark Foundation. *Wireshark Sample Captures*. Available at: https://wiki.wireshark.org/SampleCaptures [Accessed 2026].

[15] IBM. *IBM Plex Typeface*. Available at: https://www.ibm.com/plex/ [Accessed 2026].